

SLE-3: Architectural Design (Full C4 Model)

Course: 02AML204 — Introduction to Artificial Intelligence

PRN: 25UAM085

Name: Sharvil S. Chavan

Division: B

Date: 03 October 2026

GitHub Link (optional): <https://github.com/SharvilChavan-10>

1. System Title & Short Description

System: Maze Solver System (BFS / DFS) — the same maze path-finding system that was profiled in SLE-2.

The system generates grid mazes with walls, a single start cell and a single goal cell, and solves them with two uninformed search algorithms, Breadth-First Search (BFS) and Depth-First Search (DFS). For every run it records the execution time, the number of nodes expanded and the length of the path found. It solves the problem of comparing search strategies fairly on the same maze (20×20, 40×40 and 70×70). SLE-1 built a rule-based agent, SLE-2 measured the search code, and this SLE-3 documents the architecture behind it.

2. Context Diagram (Level 1)



Fig. 1 — C4 Level 1: the Maze Solver System and the world around it.

The Student / Operator gives the system a maze configuration (size, start and goal) and receives the path, the timing and node-count statistics and the result charts. The system relies on two outside pieces only: the Python runtime (for `time.perf_counter` and `collections`) and Matplotlib (to draw the maze and the result charts). There is no database, network or other external service.

3. Container Diagram (Level 2)

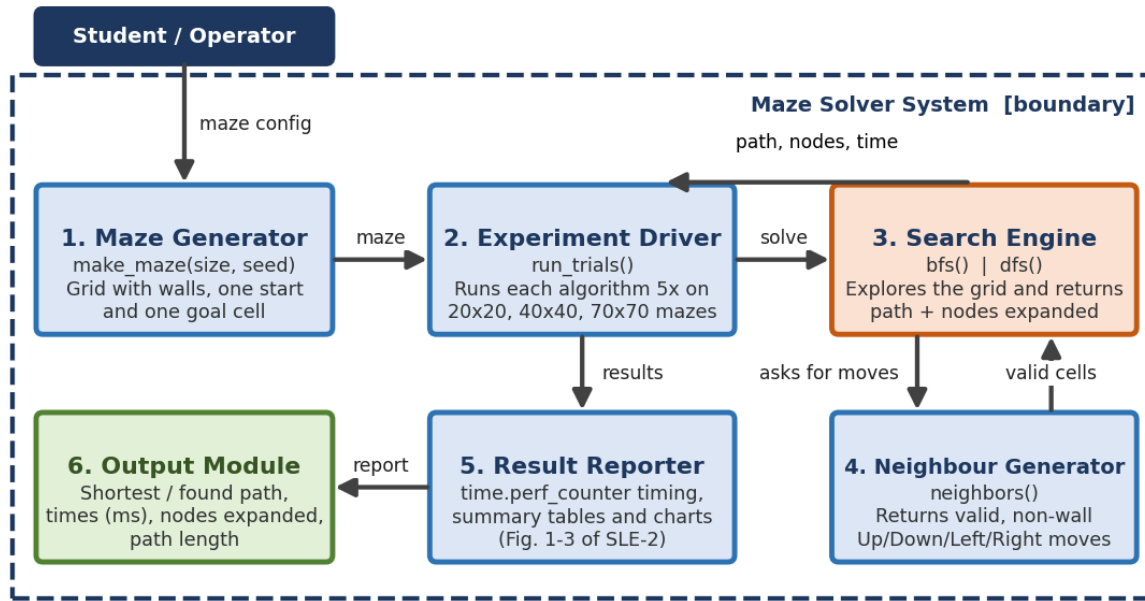


Fig. 2 — C4 Level 2: the six main building blocks of the system.

- **Maze Generator:** creates the grid maze with walls, start and goal using `make_maze()` and a seed so the maze can be repeated.
- **Experiment Driver:** controls the experiment; runs each algorithm 5 times on each maze size and passes the same maze to both.
- **Search Engine:** runs BFS or DFS on the maze and returns the path found and the number of nodes expanded.
- **Neighbour Generator:** gives the Search Engine the valid, non-wall moves from any cell, so both algorithms share one move rule.
- **Result Reporter:** times each run with `time.perf_counter`, builds the summary tables and draws the charts used in SLE-2.
- **Output Module:** the final result shown to the user: path, time in ms, nodes expanded and path length.

4. Component Diagram (Level 3)

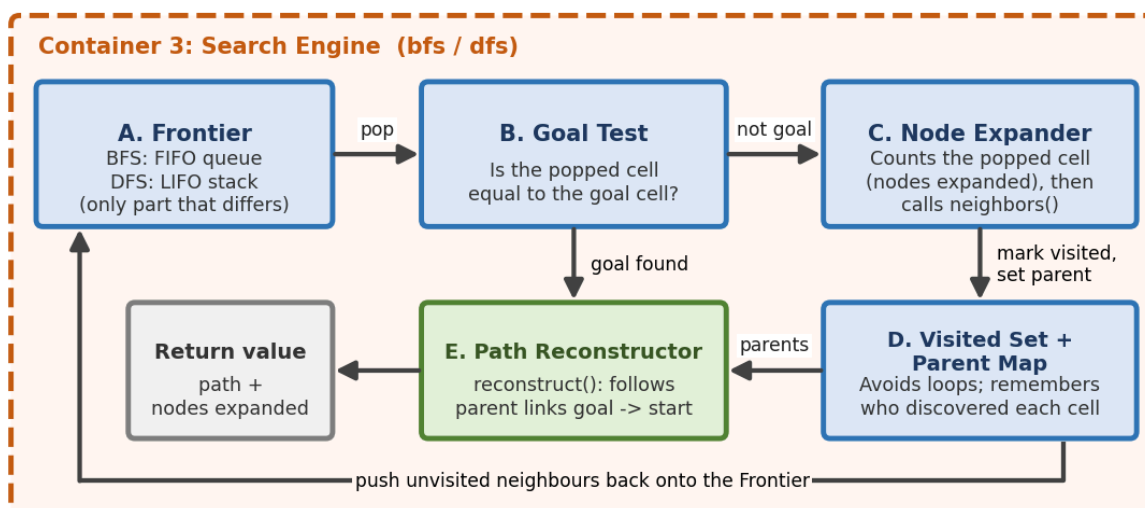


Fig. 3 — C4 Level 3: components inside the Search Engine container only.

The Search Engine is the most important container, so only its internals are drawn. The Frontier holds the cells waiting to be explored (a FIFO queue for BFS, a LIFO stack for DFS). The next cell is popped and checked by the Goal Test; if it is not the goal, the Node Expander counts it and asks for its neighbours. New cells are recorded in the Visited Set + Parent Map, which prevents loops, and unvisited neighbours go back to the Frontier. When the goal is found, the Path Reconstructor follows the parent links back to the start.

5. Code Level Overview (Level 4)

Main functions of the implementation (names and one-line responsibilities only):

Name	Kind	Responsibility (container)
<code>make_maze()</code>	function	Builds a grid maze with walls, one start and one goal cell; seeded so runs are repeatable (Maze Generator).
<code>neighbors()</code>	function	Returns the valid, non-wall Up/Down/Left/Right cells of a given cell (Neighbour Generator).
<code>bfs()</code>	function	Breadth-first search using a FIFO queue; returns the shortest path and nodes expanded (Search Engine).
<code>dfs()</code>	function	Depth-first search using a LIFO stack; returns a path and nodes expanded (Search Engine).
<code>reconstruct()</code>	function	Walks the parent map from goal back to start and reverses it to give the path (Path Reconstructor).
<code>run_trials()</code>	function	Runs each algorithm 5 times per maze size and times it with <code>time.perf_counter</code> (Experiment Driver).

6. Design Decisions

- Generator, neighbours, search and reporting are separate containers, so BFS and DFS run on the same maze with the same move rule. The only difference between them is the Frontier (queue vs stack), which keeps the SLE-2 comparison fair.
- A visited set with a parent map lets the path be rebuilt at the end, instead of storing a full path inside every frontier entry.
- Timing is kept in the Experiment Driver / Result Reporter and the node counter inside the search, so measuring does not change the search logic.
- There is no Heuristic container because BFS and DFS are uninformed; an A* version could later be added to the Search Engine as a new component.

7. AI Contribution Note

- **AI tools used:** Claude.
- **What AI helped with:** suggested the C4 structure for the maze system, drafted the four levels, produced the first versions of the diagrams and wrote the draft explanations from my SLE-1 and SLE-2 reports.
- **What I did myself:** chose the Maze Solver as the system, reviewed every diagram and description against my SLE-2 code (`maze_search.py`), and checked that the container and function names match what I implemented.

8. Conclusion

C4 showed me the same system at four levels of detail: Context shows who uses it, Container shows its main parts, Component opens up the Search Engine, and Code lists the functions that implement it. Drawing it made it clear that BFS and DFS share almost everything and differ only in how the Frontier is handled, which explains the SLE-2 results (BFS finds the shortest path, DFS expands fewer nodes). I also learned that a small, readable diagram with a few boxes explains a design better than a crowded one.